

Aula 10

Ambientes Distribuídos e Escaláveis

Disponibilidade, resiliência, balanceamento, observabilidade e consistência

Recorte da aula

Esta aula cobre a Aula 10 da trilha: Ambientes distribuídos e escaláveis: disponibilidade, resiliência, balanceamento, observabilidade e consistência. No edital, o foco direto é o item 1.17 - Ambientes distribuídos e escaláveis, com conexões instrumentais com microsserviços, APIs, mensageria, Docker, Kubernetes, Rancher e integração de sistemas.

Estratégia de prova FGV

A FGV tende a cobrar este tema por distinções: disponibilidade versus resiliência; liveness versus readiness; escalabilidade vertical versus horizontal; balanceamento versus service discovery; logs versus métricas versus traces; consistência forte versus consistência eventual; e o erro de tratar qualquer arquitetura distribuída como automaticamente escalável, resiliente e observável.

1. Identificação da aula e aderência ao edital

A aula trata do item 1.17 do edital: ambientes distribuídos e escaláveis. Em Arquitetura de Sistemas e Integração, esse item não é uma aula de infraestrutura pura. Ele pergunta: como projetar sistemas que continuam funcionando, escalam sob demanda, suportam falhas parciais, permitem diagnóstico e lidam com a dificuldade de manter dados consistentes quando componentes se comunicam por rede?

Para o seu contexto de Java/Quarkus, Angular, APIs e ambiente corporativo, pense em uma aplicação web ou mobile que chama um API Gateway; por trás dele existem serviços Java, bancos, filas RabbitMQ, integrações externas, pods em Kubernetes, dashboards e alertas. A prova pode perguntar qual componente resolve qual problema e qual trade-off aparece quando uma decisão arquitetural é tomada.

Objetivo da aula: transformar a visão prática de arquitetura distribuída em acerto de prova, evitando respostas absolutas como 'Kubernetes garante disponibilidade', 'retry sempre melhora resiliência', 'observabilidade é apenas log' ou 'consistência eventual é erro'.

2. Vocabulário essencial

Termo	Definição objetiva	Pegadinha FGV
Ambiente distribuído	Conjunto de componentes que executam em nós/processos diferentes e se comunicam por rede.	Tratar rede como detalhe irrelevante. Rede falha, atrasa, duplica, particiona e exige desenho explícito.
Escalabilidade	Capacidade de aumentar ou reduzir capacidade de processamento sem redesenhar todo o sistema.	Confundir escalar com apenas comprar máquina maior.
Disponibilidade	Capacidade de o serviço responder corretamente dentro de limites aceitáveis de tempo e erro.	Achar que disponibilidade é só servidor ligado.
Resiliência	Capacidade de absorver falhas, degradar de forma controlada e recuperar operação.	Confundir com ausência de falhas.
Balanceamento	Distribuição de tráfego entre instâncias, nós ou caminhos disponíveis.	Confundir com service discovery ou com escalabilidade automática.
Observabilidade	Capacidade de inferir o estado interno do sistema por sinais externos, como logs, métricas e traces.	Reduzir observabilidade a logs soltos.
Consistência	Grau de concordância/percepção sobre o estado dos dados entre réplicas, caches, bancos e serviços.	Exigir consistência forte em todo cenário sem avaliar custo de latência e disponibilidade.

3. Modelo mental de sistema distribuído

Um sistema distribuído nasce quando a aplicação deixa de ser um único processo com acesso direto a tudo. Em vez disso, o fluxo passa por rede, gateways, serviços, filas, bancos e integrações. Isso aumenta autonomia e escalabilidade, mas também cria falhas parciais: um serviço pode estar de pé enquanto outro está indisponível; uma mensagem pode ser entregue duas vezes; uma réplica pode estar desatualizada; um dashboard pode parecer verde enquanto um usuário específico está impactado.

Angular/mobile
-> API Gateway / Ingress / Load Balancer
-> serviço-processos (pods Java/Quarkus ou Spring)
-> serviço-documentos
-> RabbitMQ -> serviço-notificacao
-> banco relacional / cache / API externa
-> telemetria: logs + métricas + traces + alertas

O ponto de prova é perceber que cada camada resolve um problema diferente. Gateway não é broker; broker não é banco; Kubernetes não corrige modelagem errada; observabilidade não nasce automaticamente porque a aplicação foi containerizada; e consistência forte não é gratuita em ambiente replicado.

4. Disponibilidade: mais do que 'estar no ar'

Disponibilidade é a capacidade de o sistema estar acessível e útil para o usuário. Em prova, cuidado: um serviço pode estar 'up' no health check e ainda assim estar inútil se depende de banco, fila, autenticação ou API externa indisponível. Disponibilidade envolve redundância, remoção de ponto único de falha, monitoramento, failover, capacidade, tempo de resposta e tratamento de degradação.

Nível	Indisponibilidade aproximada por ano	Leitura de prova
99%	3,65 dias	Pode ser aceitável para sistema pouco crítico, mas ruim para serviços essenciais.
99,9%	8,76 horas	Um 'nove' a mais reduz bastante a janela de falha.
99,99%	52,6 minutos	Exige arquitetura, operação e automação mais maduras.

Nível	Indisponibilidade aproximada por ano	Leitura de prova
99,999%	5,26 minutos	Custo e complexidade sobem muito; não é obtido apenas com framework.

Não confunda SLA, SLO e SLI. SLA é compromisso formal de nível de serviço; SLO é objetivo interno ou acordado de confiabilidade; **SLI é indicador que mede o serviço, como taxa de erro, latência p95/p99, disponibilidade de endpoint ou percentual de requisições bem-sucedidas.**

5. Resiliência: falhar bem e recuperar rápido

Resiliência é a capacidade de o sistema continuar prestando serviço, ainda que de modo degradado, diante de falhas. Em arquitetura distribuída, falha parcial é a regra: uma instância pode travar, um banco pode ficar lento, uma fila pode acumular, uma API externa pode responder 500 ou demorar 30 segundos. A resposta correta não é 'tentar infinitamente'; é desenhar limites e políticas.

Técnica	Função	Erro comum
Timeout	Impede que uma chamada espere indefinidamente.	Usar timeout alto demais e prender threads/conexões.
Retry	Tenta novamente falhas transitórias.	Aplicar retry em operação não idempotente e duplicar efeitos.
Backoff	Aumenta intervalo entre tentativas.	Fazer retries agressivos e piorar a sobrecarga.
Circuit breaker	Abre o circuito após falhas e evita chamadas repetidas a dependência degradada.	Tratar como mecanismo de autenticação ou balanceamento.
Bulkhead	Isola recursos para que uma falha não consuma todo o sistema.	Achar que uma fila única infinita resolve tudo.
Fallback	Resposta alternativa/degradada quando o caminho principal falha.	Mascarar erro crítico como sucesso sem rastreabilidade.
Idempotência	Permite repetir uma operação sem multiplicar efeitos indevidos.	Ignorar em mensageria e APIs com retry.

Regra de prova

Retry sem timeout, sem backoff, sem limite e sem idempotência pode piorar o incidente. Circuit breaker não recupera a dependência; ele evita que a falha se propague indefinidamente. Fallback não deve esconder perda de dados ou violação de regra negocial.

6. Balanceamento e escalabilidade

Escalar significa ajustar capacidade. Escalabilidade vertical aumenta recursos de uma instância: mais CPU, memória, I/O. Escalabilidade horizontal aumenta o número de instâncias ou réplicas. Em sistemas web e microserviços, a escalabilidade horizontal é comum porque permite distribuir tráfego, substituir instâncias e reduzir dependência de uma máquina específica.

Conceito	Resolve	Não resolve sozinho
Load balancer	Distribui requisições entre instâncias disponíveis.	Não corrige código lento, banco saturado ou endpoint sem idempotência.
Service discovery	Permite localizar instâncias dinamicamente.	Não é, por si só, autenticação, gateway ou observabilidade.
API Gateway	Centraliza entrada, roteamento e políticas transversais.	Não deve concentrar regra negocial profunda.
HPA	Aumenta/reduz réplicas de pods conforme métricas.	Não resolve gargalo externo, como banco único saturado.
Stateless service	Facilita réplicas e substituição de instâncias.	Não significa ausência de persistência; significa não depender de estado local da instância.

No Kubernetes, **Service fornece um ponto estável para acessar pods que podem nascer, morrer ou mudar de IP. Probes de readiness evitam enviar tráfego para pod ainda não pronto; probes de liveness ajudam a reiniciar containers travados; startup probes protegem aplicações que demoram a iniciar.** HPA ajusta número de réplicas, mas a métrica escolhida precisa representar o gargalo real.

7. Observabilidade: logs, métricas e traces

Monitoramento coleta, processa, agrega e apresenta dados quantitativos em tempo real sobre o sistema. Observabilidade vai além: permite investigar por que algo aconteceu em um sistema distribuído. Para a prova, a tríade essencial é: logs registram eventos; métricas medem comportamento ao longo do tempo; traces mostram o caminho de uma requisição entre componentes.

Sinal	Pergunta que responde	Exemplo
Logs	O que aconteceu neste processo/serviço?	Erro ao consultar documento, usuário X, correlation-id abc.
Métricas	O comportamento agregado piorou?	Taxa de erro, latência p95, uso de CPU, fila acumulada.
Traces	Por onde passou esta requisição?	Gateway -> processo-service -> documento-service -> banco.
Baggage/contexto	Que contexto acompanha a execução?	Tenant, usuário, unidade, correlation-id, origem.

O Google SRE populariza a ideia dos quatro sinais de ouro: latência, tráfego, erros e saturação. Em uma prova FGV, um cenário com dashboard cheio de CPU e memória, mas sem métricas de erro/latência por endpoint, está incompleto. Em ambiente distribuído, correlation ID e tracing são decisivos para seguir uma chamada que atravessa gateway, serviços e mensageria.

8. Consistência em sistemas distribuídos

Consistência é uma das áreas mais traiçoeiras. Em sistema monolítico com um banco transacional, o candidato costuma raciocinar como se toda alteração fosse instantânea e global. **Em sistema distribuído, dados podem estar em serviços diferentes, bancos diferentes, caches, réplicas ou filas. Nem tudo fica consistente ao mesmo tempo.**

Modelo	Ideia	Quando aparece
Consistência forte	Leituras refletem a escrita mais recente conforme uma ordem única percebida.	Operações críticas que não toleram leitura obsoleta.
Consistência eventual	Réplicas/serviços convergem com o tempo se novas atualizações cessarem.	Eventos, caches, buscas, notificações e integrações assíncronas.
Read-your-writes	Usuário lê o que ele mesmo acabou de gravar.	Experiência de usuário após cadastro/alteração.
Saga	Sequência de transações locais com compensações em caso de falha.	Processos distribuídos sem transação ACID única.
Outbox	Grava mudança e evento na mesma transação local para publicar depois com segurança.	Evitar perder evento após commit do banco.

CAP theorem costuma aparecer de modo simplificado: diante de partição de rede, há trade-off entre consistência e disponibilidade. Porém, para prova de Analista, o mais importante é não transformar CAP em slogan. Sistemas reais também lidam com latência, falha de nó, saturação, filas, caches e níveis diferentes de consistência. A resposta correta costuma ser a que reconhece trade-off, não a que diz que 'um banco é simplesmente CP ou AP' sem contexto.

9. Exemplo prático integrado

Cenário: uma aplicação Angular consulta processos e documentos. O gateway autentica, roteia e aplica rate limit. O processo-service em Quarkus consulta banco próprio. O documento-service é uma dependência interna. Notificações são enviadas por RabbitMQ. Tudo roda em Kubernetes, com múltiplas réplicas.

Problema observado	Boa leitura arquitetural	Resposta ruim
Uma réplica travou	Liveness probe pode reiniciar; readiness impede tráfego para pod não pronto.	Aumentar timeout indefinidamente.
API externa ficou lenta	Timeout, circuit breaker, fallback e fila quando cabível.	Retry infinito em todos os clientes.
Fila cresceu muito	Métrica de backlog, consumidores, throughput, DLQ, limites e backpressure.	Ignorar porque HTTP ainda responde 200.
Usuário não vê imediatamente notificação	Pode ser consistência eventual/evento assíncrono esperado.	Concluir que há erro de transação obrigatoriamente.
Erro só ocorre em um caminho	Trace distribuído e correlation-id ajudam a localizar serviço/dependência.	Ler apenas log local do frontend.

10. Trade-offs de alto rendimento

Decisão	Benefício	Custo ou risco
Escalar horizontalmente serviços stateless	Mais capacidade e tolerância a falha de instância.	Exige estado fora da instância e observabilidade por réplica.
Adicionar cache	Reduz latência e carga no backend.	Pode entregar dado obsoleto e exige invalidação.
Usar mensageria	Desacopla e absorve picos.	Introduz atraso, ordem, duplicidade e necessidade de idempotência.
Centralizar entrada em gateway	Governança, segurança e roteamento padronizados.	Pode virar gargalo ou monólito de borda.
Aumentar retry	Ajuda em falha transitória.	Pode amplificar incidente e duplicar efeitos.

Decisão	Benefício	Custo ou risco
Consistência forte	Reduz ambiguidade de leitura/escrita.	Pode aumentar latência e reduzir disponibilidade em falhas de rede.

11. Pegadinhas FGV

- Ambiente distribuído não é automaticamente escalável: se todas as chamadas dependem de um banco único saturado, adicionar réplicas de API pode não resolver.
- Disponibilidade não é o mesmo que resiliência: estar disponível é responder; ser resiliente é absorver falhas e recuperar.
- Liveness probe não deve verificar todas as dependências externas; se fizer isso, pode reiniciar pods saudáveis por falha de terceiros.
- Readiness probe indica se o pod deve receber tráfego; liveness indica se deve ser reiniciado; startup protege inicialização lenta.
- Balanceador distribui tráfego; service discovery localiza instâncias; API Gateway aplica entrada, roteamento e políticas transversais.
- Observabilidade não é apenas log; métricas e traces são centrais em arquitetura distribuída.
- Circuit breaker não é autenticação, autorização, retry nem fila.
- Retry sem idempotência pode duplicar efeitos: cobrança, protocolo, notificação ou movimentação.
- Consistência eventual não é inconsistência permanente; é convergência controlada com tempo e regras.
- CAP não deve ser usado como slogan absoluto; a prova pode exigir reconhecer trade-offs práticos de latência, disponibilidade e consistência.

12. Questões autorais - padrão FGV

Resolva as 20 questões antes de consultar o gabarito. As questões são autorais e calibradas para discriminar conceitos próximos, sem copiar questões reais.

Questão 1. Em um sistema distribuído usado por servidores e cidadãos, a equipe deseja que a aplicação continue prestando serviço mesmo quando uma instância de backend falhar. A medida mais alinhada a disponibilidade e resiliência é:

- A) concentrar todas as sessões em memória local de uma única instância, para evitar divergência.
- B) remover health checks, pois reinicializações automáticas podem ocultar falhas.
- C) implantar múltiplas réplicas stateless, com balanceamento, probes adequadas e estratégia de degradação.
- D) aumentar indefinidamente o timeout de todos os clientes.
- E) substituir logs por comentários no código para reduzir custo operacional.

Questão 2. Em Kubernetes, uma aplicação demora dois minutos para inicializar, mas depois funciona normalmente. A equipe não quer que o container seja morto durante a inicialização lenta, nem que receba tráfego antes de estar pronto. A combinação conceitualmente mais adequada envolve:

- A) apenas liveness probe, verificando todas as dependências externas.
- B) startup probe para proteger a inicialização e readiness probe para controlar entrada de tráfego.
- C) somente Horizontal Pod Autoscaler, pois autoscaling substitui health check.
- D) apenas logs de aplicação, pois probes são mecanismos de observabilidade.
- E) circuit breaker no frontend Angular, pois probe é função do cliente.

Questão 3. A respeito de escalabilidade horizontal, assinale a alternativa correta.

- A) Consiste em aumentar o número de instâncias ou réplicas para distribuir carga.
- B) Consiste exclusivamente em aumentar CPU e memória da mesma máquina.
- C) Só é possível em aplicações stateful com sessão local obrigatória.
- D) Elimina a necessidade de banco de dados e de observabilidade.
- E) É sinônimo de cache distribuído.

Questão 4. Em uma arquitetura com API Gateway, Service Discovery e Load Balancer, assinale a leitura mais adequada.

- A) O API Gateway centraliza entrada, políticas e roteamento; o service discovery localiza instâncias; o balanceador escolhe ou distribui tráfego entre destinos disponíveis.
- B) O service discovery autentica usuários finais e aplica CORS.
- C) O balanceador armazena eventos comerciais de forma durável.
- D) O API Gateway substitui banco de dados e message broker.
- E) Os três termos são sinônimos em arquitetura distribuída.

Questão 5. Um contrato formal promete que 99,9% das requisições válidas a uma API serão respondidas com sucesso em até determinado tempo. A métrica real usada para acompanhar percentual de sucesso e latência é chamada, respectivamente, de:

- A) cache e gateway.
- B) saga e outbox.
- C) broker e exchange.
- D) cluster e namespace.
- E) SLA/SLO como compromisso ou objetivo; SLI como indicador mensurável que acompanha o comportamento real.

Questão 6. Uma API de protocolo processual chama um serviço externo que fica instável. A equipe adiciona retries ilimitados em todas as chamadas, sem timeout, sem backoff e sem idempotência. Sob a ótica de resiliência, a decisão é:

- A) correta, porque retry ilimitado sempre aumenta disponibilidade.
- B) correta apenas se a aplicação estiver em Kubernetes.
- C) inadequada, pois pode amplificar a falha, prender recursos e duplicar efeitos.
- D) obrigatória, pois circuit breaker só funciona com retry infinito.
- E) irrelevante, pois resiliência depende apenas do banco.

Questão 7. Uma equipe deseja investigar por que uma requisição de consulta processual ficou lenta ao atravessar gateway, serviço de processos, serviço de documentos e banco. O sinal de observabilidade mais

apropriado para visualizar o caminho da requisição é:

- A) CPU média do nó Kubernetes.
- B) trace distribuído, preferencialmente com correlation-id/contexto propagado.
- C) percentual de cobertura de testes unitários.
- D) número de réplicas do Deployment.
- E) tamanho da imagem Docker.

Questão 8. Assinale a alternativa que distingue corretamente logs, métricas e traces.

- A) Logs medem exclusivamente CPU; métricas mostram código-fonte; traces compilam imagens Docker.
- B) Logs e traces são sinônimos; métricas são usadas apenas em bancos relacionais.
- C) Traces são comentários no código; logs são dashboards; métricas são mensagens RabbitMQ.
- D) Logs registram eventos; métricas capturam medidas agregáveis no tempo; traces mostram o percurso de uma requisição entre componentes.
- E) Logs substituem completamente métricas e traces em microsserviços.

Questão 9. Um serviço grava a movimentação processual e publica evento para serviço de notificações. O usuário pode ver a movimentação imediatamente, mas a notificação pode chegar alguns segundos depois. A melhor interpretação é:

- A) pode ser um caso aceitável de consistência eventual em fluxo assíncrono, desde que haja contrato, monitoramento e tratamento de falhas.
- B) é obrigatoriamente erro grave, pois todos os efeitos devem ser síncronos.
- C) prova que o sistema não possui banco de dados.
- D) demonstra que o API Gateway faz mensageria interna.
- E) indica ausência de autenticação SSO.

Questão 10. Sobre o uso do teorema CAP em questões de arquitetura distribuída, assinale a alternativa mais adequada.

- A) Todo sistema distribuído escolhe livremente C, A e P ao mesmo tempo, sem trade-off.
- B) Em presença de partição de rede, há trade-off entre consistência e disponibilidade; porém, em sistemas reais, é preciso analisar também latência, falhas, escopo e garantias concretas.
- C) CAP significa que bancos relacionais nunca podem ser consistentes.
- D) CAP se aplica apenas a frontend Angular.
- E) A existência de Kubernetes elimina o problema de consistência.

Questão 11. Em um fluxo de atualização de dados e publicação de evento, a equipe grava a alteração no banco, mas falha antes de publicar a mensagem no broker. Um padrão frequentemente usado para reduzir esse risco é:

- A) sticky session obrigatória.
- B) liveness probe com TCP.
- C) paginação de resposta REST.
- D) replicação de frontend.
- E) outbox, gravando evento e mudança de estado na mesma transação local para publicação posterior controlada.

Questão 12. Um serviço é descrito como stateless em uma arquitetura Kubernetes. Isso significa que:

- A) ele não pode acessar banco de dados.
- B) ele não pode receber requisições HTTP.
- C) ele não depende de estado local da instância para continuar funcionando corretamente, facilitando réplicas e substituição de pods.
- D) ele obrigatoriamente usa banco em memória.
- E) ele dispensa autenticação e autorização.

Questão 13. Em um desenho de alta disponibilidade, o objetivo é reduzir ponto único de falha. Assinale a alternativa correta.

- A) Usar uma única instância de gateway sem redundância aumenta disponibilidade por simplificar a topologia.
- B) Remover métricas melhora disponibilidade por reduzir consumo de CPU.
- C) Confiar apenas no frontend Angular é suficiente para tolerância a falha de backend.
- D) Redundância, balanceamento, failover, health checks e automação de recuperação ajudam a reduzir impacto de falhas.

E) Consistência eventual significa que não há necessidade de backup ou recuperação.

Questão 14. Um serviço começa a receber volume superior ao suportado. Para evitar colapso, a arquitetura pode usar limites de taxa, filas com capacidade, descarte controlado ou sinalização para produtores reduzirem envio. Esse conjunto se relaciona mais diretamente a:

- A) backpressure e controle de fluxo.
- B) normalização de banco.
- C) herança de classes.
- D) view server-side.
- E) geração de chaves primárias.

Questão 15. Em continuidade e resiliência, RTO e RPO significam, respectivamente:

- A) tempo máximo aceitável para recuperação do serviço e perda máxima aceitável de dados no ponto de recuperação.
- B) taxa de objetos por requisição e política de origem de CORS.
- C) número de réplicas e percentual de CPU.
- D) rota de gateway e nome do serviço no discovery.
- E) protocolo de eventos e formato de payload.

Questão 16. Um Horizontal Pod Autoscaler aumenta o número de réplicas de uma API porque a CPU subiu. Ainda assim, a latência permanece alta, pois o gargalo real está no banco compartilhado. A leitura correta é:

- A) HPA substitui tuning de banco e deve resolver qualquer gargalo.
- B) O problema está obrigatoriamente no Angular.
- C) Escalar camada de aplicação não elimina gargalos em dependências compartilhadas.
- D) A única solução é remover logs.
- E) Readiness probe deve ser removida para aumentar tráfego.

Questão 17. Analise as afirmativas: I. Observabilidade permite inferir o estado interno por sinais como logs, métricas e traces. II. Em ambiente distribuído, falha parcial é possível mesmo quando parte do sistema está saudável. III. Consistência eventual significa que os dados jamais convergem. Está correto o que se afirma em:

- A) I e II, apenas.
- B) I, apenas.
- C) II e III, apenas.
- D) I, II e III.
- E) III, apenas.

Questão 18. Relacione os conceitos às funções. 1. Readiness probe 2. Liveness probe 3. Trace distribuído 4. Circuit breaker () Evita enviar tráfego a instância ainda não pronta. () Detecta container travado e pode levar à reinicialização. () Mostra o caminho da requisição entre serviços. () Evita chamadas repetidas a dependência falhando. A sequência correta é:

- A) 2 - 1 - 3 - 4.
- B) 1 - 2 - 3 - 4.
- C) 1 - 3 - 2 - 4.
- D) 4 - 2 - 3 - 1.
- E) 3 - 2 - 1 - 4.

Questão 19. Uma equipe possui dashboards com CPU, memória e quantidade de pods, mas não mede erro por endpoint, latência p95/p99, saturação de fila nem traces de requisição. A melhor avaliação é:

- A) a observabilidade está completa, pois CPU e memória bastam.
- B) tracing é desnecessário em sistemas distribuídos.
- C) métricas de negócio e erro devem ficar apenas no banco.
- D) a visão é incompleta, pois faltam sinais úteis para comportamento percebido pelo usuário e fluxo distribuído.
- E) logs devem ser removidos para reduzir custo.

Questão 20. Assinale a alternativa INCORRETA sobre ambientes distribuídos e escaláveis.

- A) Escalar horizontalmente uma API sempre garante consistência forte entre todos os dados do sistema, independentemente do banco, cache ou mensageria.
- B) Um API Gateway pode ajudar a centralizar entrada e políticas transversais, mas pode virar gargalo se mal desenhado.

- C) Mensageria pode desacoplar serviços, mas exige atenção a duplicidade, ordenação, DLQ, idempotência e consistência eventual.
- D) Readiness e liveness probes possuem propósitos distintos em Kubernetes.
- E) Logs, métricas e traces cumprem papéis complementares em observabilidade.

13. Gabarito resumido

Questão	Gabarito	Questão	Gabarito	Questão	Gabarito	Questão	Gabarito
1	C	6	C	11	E	16	C
2	B	7	B	12	C	17	A
3	A	8	D	13	D	18	B
4	A	9	A	14	A	19	D
5	E	10	B	15	A	20	A

Gabarito em sequência: 1C - 2B - 3A - 4A - 5E - 6C - 7B - 8D - 9A - 10B - 11E - 12C - 13D - 14A - 15A - 16C - 17A - 18B - 19D - 20A

14. Comentários completos

Questão 1 - Gabarito: C

A correta é C. Múltiplas réplicas stateless, balanceamento, health checks e degradação controlada atacam disponibilidade e resiliência. A erra ao prender sessão em uma instância, criando acoplamento e ponto único de falha. B remove mecanismos de detecção. D transforma lentidão em espera indefinida. E confunde observabilidade com documentação de código.

Questão 2 - Gabarito: B

A correta é B. Startup probe protege inicialização lenta; readiness impede tráfego antes de prontidão. A é perigosa quando usa liveness para dependências externas; isso pode reiniciar container saudável. C confunde autoscaling com saúde. D reduz probes a logs. E desloca responsabilidade para o frontend.

Questão 3 - Gabarito: A

A correta é A. Escalabilidade horizontal aumenta instâncias ou réplicas. B descreve escalabilidade vertical. C erra porque sessão local dificulta escalar horizontalmente. D exagera: escalar não elimina banco nem observabilidade. E confunde escala com cache.

Questão 4 - Gabarito: A

A correta é A. Gateway, discovery e balanceamento são conceitos associados, mas não idênticos. B atribui autenticação/CORS ao discovery. C atribui durabilidade de eventos ao balanceador. D transforma gateway em banco/broker. E elimina distinções fundamentais de prova.

Questão 5 - Gabarito: E

A correta é E. SLA/SLO representam compromisso ou objetivo de serviço, enquanto SLI é o indicador mensurável usado para avaliar o comportamento real. As demais alternativas misturam conceitos de arquitetura, mensageria, cluster ou gateway que não expressam essa relação.

Questão 6 - Gabarito: C

A correta é C. Retry ilimitado sem timeout, backoff e idempotência pode amplificar incidente, esgotar threads/conexões e duplicar efeitos. A é a pegadinha clássica do 'retry sempre melhora'. B condiciona indevidamente a Kubernetes. D confunde circuit breaker com retry infinito. E restringe resiliência ao banco.

Questão 7 - Gabarito: B

A correta é B. Trace distribuído mostra o caminho da requisição entre componentes, especialmente com correlation-id. A é métrica útil, mas agregada e local ao nó. C mede qualidade de testes. D é capacidade declarada. E é artefato de empacotamento.

Questão 8 - Gabarito: D

A correta é D. Logs, métricas e traces são sinais complementares. A, B e C trocam conceitos. E erra porque logs não substituem métricas nem tracing; em microserviços, depender só de log local dificulta diagnóstico.

Questão 9 - Gabarito: A

A correta é A. Notificação assíncrona pode ser eventualmente consistente. Isso é aceitável quando o contrato é claro e há monitoramento, DLQ/reprocessamento e idempotência. B exige sincronia absoluta. C, D e E deslocam o problema para banco, gateway ou autenticação.

Questão 10 - Gabarito: B

A correta é B. CAP é útil como noção de trade-off em partição de rede, mas não deve ser slogan. Sistemas reais também sofrem com latência, falha de nó, saturação e diferentes garantias. A nega trade-off. C, D e E são distorções.

Questão 11 - Gabarito: E

A correta é E. Outbox grava alteração e evento de forma transacional local e permite publicação posterior controlada, reduzindo perda de evento após commit. A fala de sessão. B de saúde. C de API REST. D de frontend.

Questão 12 - Gabarito: C

A correta é C. Stateless significa não depender de estado local da instância, facilitando réplicas e substituição. A é falsa: serviço stateless pode persistir dados em banco externo. B, D e E inventam restrições.

Questão 13 - Gabarito: D

A correta é D. Alta disponibilidade requer redundância, balanceamento, failover, health checks e automação. A cria ponto único de falha. B remove sinais necessários. C superestima frontend. E confunde consistência eventual com ausência de proteção de dados.

Questão 14 - Gabarito: A

A correta é A. Backpressure e controle de fluxo evitam que produtores ou clientes sobrecarreguem consumidores. B, C, D e E pertencem a banco, OO, apresentação ou persistência, sem relação direta com saturação distribuída.

Questão 15 - Gabarito: A

A correta é A. RTO é tempo máximo aceitável para recuperação; RPO é perda máxima aceitável de dados em relação ao ponto de recuperação. B, C, D e E usam siglas e termos de outros temas como distração.

Questão 16 - Gabarito: C

A correta é C. Escalar pods de API não elimina gargalo em banco, cache, fila ou serviço externo. A superestima HPA. B culpa o frontend sem base. D e E removem mecanismos úteis sem resolver a dependência saturada.

Questão 17 - Gabarito: A

A correta é A. I e II estão corretas: observabilidade usa sinais; falha parcial é típica de sistemas distribuídos. III está errada porque consistência eventual pressupõe convergência, não divergência permanente.

Questão 18 - Gabarito: B

A correta é B: readiness evita tráfego a instância não pronta; liveness detecta travamento; trace mostra caminho; circuit breaker evita chamadas repetidas a dependência falhando. As demais sequências trocam readiness, liveness ou trace.

Questão 19 - Gabarito: D

A correta é D. CPU, memória e pods ajudam, mas não bastam. Sem erro por endpoint, latência percentil, saturação e tracing, a equipe enxerga infraestrutura, mas pouco do comportamento percebido pelo usuário e do fluxo distribuído. A, B, C e E são reduções indevidas.

Questão 20 - Gabarito: A

A incorreta é A. Escalar API horizontalmente não garante consistência forte; isso depende de banco, cache, replicação, mensageria e contrato de dados. B, C, D e E estão corretas: gateway, mensageria, probes e observabilidade têm papéis e trade-offs próprios.

15. Checklist final de cobertura

- Diferenciar disponibilidade, resiliência, escalabilidade e observabilidade.
- Diferenciar escalabilidade vertical e horizontal, além de saber o limite de autoscaling.
- Entender load balancer, API Gateway e Service Discovery sem fundir responsabilidades.
- Aplicar timeout, retry, backoff, circuit breaker, bulkhead, fallback e idempotência em cenários concretos.
- Distinguir liveness, readiness e startup probes em Kubernetes.
- Reconhecer logs, métricas, traces e correlation-id como base de observabilidade distribuída.
- Entender consistência forte, consistência eventual, saga, outbox e trade-offs de CAP sem slogan superficial.

16. Referências técnicas consultadas

- Edital TJSC 2026 - Analista de Sistemas - FGV: <https://conhecimento.fgv.br/concursos/tjscservidor26>
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- Google SRE Book - Monitoring Distributed Systems: <https://sre.google/sre-book/monitoring-distributed-systems/>
- Kubernetes Documentation - Liveness, Readiness and Startup Probes: <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>
- Kubernetes Documentation - Horizontal Pod Autoscaling: <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>
- OpenTelemetry Documentation - Observability and signals: <https://opentelemetry.io/docs/>
- OpenTelemetry Signals: <https://opentelemetry.io/docs/concepts/signals/>

- Martin Kleppmann - Please stop calling databases CP or AP: <https://martin.kleppmann.com/2015/05/11/please-stop-calling-databases-cp-or-ap.html>
- Kleppmann - A Critique of the CAP Theorem: <https://arxiv.org/abs/1509.05393>